

```
In [1]: class ClassroomAgent:

    def __init__(self):
        self.previous_mode = None

    def act(self, temp, occupied):

        if not occupied:
            target_mode = "ECO"

        elif temp > 26:
            target_mode = "COOL"

        elif temp < 20:
            target_mode = "WARM"

        else:
            target_mode = "IDLE"

        # Check whether a new command is needed
        if target_mode == self.previous_mode:
            command = "NO COMMAND"
        else:
            command = "SEND COMMAND"

        # Remember the current mode
        previous = self.previous_mode
        self.previous_mode = target_mode

        return target_mode, command

agent = ClassroomAgent()

percepts = [
    (29, True),
    (29, True),
    (20, True),    # exact lower boundary
    (26, True),    # exact upper boundary
    (19, True),    # transition to WARM
    (29, False)    # empty room
]

for step, percept in enumerate(percepts, 1):

    previous = agent.previous_mode
    target, command = agent.act(*percept)

    print(
        step,
        "Previous:", previous,
        "Percept:", percept,
        "Target mode:", target,
```

```
) "Outcome:", command
```

```
1 Previous: None Percept: (29, True) Target mode: COOL Outcome: SEND COMMAND  
2 Previous: COOL Percept: (29, True) Target mode: COOL Outcome: NO COMMAND  
3 Previous: COOL Percept: (20, True) Target mode: IDLE Outcome: SEND COMMAND  
4 Previous: IDLE Percept: (26, True) Target mode: IDLE Outcome: NO COMMAND  
5 Previous: IDLE Percept: (19, True) Target mode: WARM Outcome: SEND COMMAND  
6 Previous: WARM Percept: (29, False) Target mode: ECO Outcome: SEND COMMAND
```

In []: